Отчёт по домашнему заданию

Аналитический пайплайн обработки данных о вакансиях: сравнение MapReduce и DuckDB

Вариант: Go (Golang) разработчик

Выполнил: студент
Лапшин Максим 466465 J3311

Содержание

1 Введение **3**

2 Этап 1. Развёртывание объектного хранилища **3**
2.1 Выбор технологии . . . 3
2.2 Запуск хранилища . . . 3

3 Этап 2. Загрузка данных **4**
3.1 Подготовка данных . . . 4
3.2 Конвертация и загрузка . . . 4

4 Этап 3. Подсчёт навыков двумя способами **4**
4.1 Фильтрация по варианту . . . 4
4.2 Собственная реализация MapReduce . . . 4
4.3 SQL-запрос через DuckDB . . . 5

5 Этап 4. Верификация результатов **6**
5.1 Методология сравнения . . . 6
5.2 Результат верификации . . . 6

6 Анализ роста числа пар и троек навыков **6**
6.1 Комбинаторное объяснение . . . 6
6.2 Наблюдения из данных . . . 6

7 Этап 5. Анализ влияния навыков на заработную плату **7**
7.1 Методология . . . 7
7.2 Результаты . . . 7
7.3 Интерпретация результатов . . . 7

8 Этап 6. Региональный анализ **8**
8.1 Методология . . . 8
8.2 Результаты . . . 8
8.3 Инсайты и наблюдения . . . 9

9 Этап 7. Профилирование и сравнение подходов **9**
9.1 Методология профилирования . . . 9
9.2 Результаты профилирования . . . 9
9.3 Анализ результатов . . . 9

10 Бонусное задание: Оптимизация расчётов **10**
10.1 Применённые оптимизации . . . 10
10.2 Сравнение “до” и “после” . . . 10

11 Выводы **10**
11.1 Технические выводы . . . 10
11.2 Бизнес-выводы . . . 11
11.3 Рекомендации для дальнейших исследований . . . 11

12 Список использованных технологий **11**

13 Заключение **12**

1 Введение

Данная работа направлена на закрепление навыков работы с объектными хранилищами и распределёнными вычислениями. В рамках задания был развёрнут полноценный аналитический пайплайн: от подготовки инфраструктуры и загрузки данных до извлечения бизнес-инсайтов и строгого профилирования ресурсов.

Цель работы: Сравнительный анализ производительности двух подходов к обработке данных (собственная реализация MapReduce vs SQL-запросы через DuckDB) на примере анализа вакансий Go-разработчиков.

Задачи:

- Развернуть объектное хранилище
- Загрузить набор данных о вакансиях
- Реализовать подсчёт навыков методами MapReduce и SQL
- Провести верификацию результатов
- Выполнить анализ влияния навыков на зарплату
- Провести региональный анализ
- Выполнить профилирование производительности

2 Этап 1. Развёртывание объектного хранилища

2.1 Выбор технологии

В качестве объектного хранилища был выбран **Moto Server** — легковесная библиотека, создающая точную копию S3-совместимого API. Выбор обусловлен:

- Полной совместимостью с boto3 (стандартная библиотека для работы с S3)
- Отсутствием необходимости в Docker (что ускорило развёртывание)
- Возможностью работы в offline-режиме

2.2 Запуск хранилища

Хранилище было запущено командой:

```
moto_server -p 9000
```

Сервер успешно запустился и готов принимать запросы по адресу `http://127.0.0.1:9000`.

3 Этап 2. Загрузка данных

3.1 Подготовка данных

Исходные данные представляли собой папку `hh` с 10000 JSON-файлов, каждый из которых содержал информацию об одной вакансии с сайта HH.ru. Структура данных включала:

- `name` — название вакансии
- `key_skills` — массив навыков (в некоторых случаях строка)
- `salary` — объект с полями `from` и `to`
- `area` — объект с информацией о регионе

3.2 Конвертация и загрузка

Для оптимизации работы с хранилищем все JSON-файлы были объединены в один файл формата **JSONL** (JSON Lines), где каждая строка представляет отдельную вакансию. Этот формат является стандартом для аналитических пайплайнов, так как позволяет читать данные частями без загрузки всего файла в память.

Результат:

- Собрано вакансий: **10 000**
- Итоговый файл: `vacancies.jsonl`
- Размер файла: **52.46 MB**
- Бакет: `hh-lake`

4 Этап 3. Подсчёт навыков двумя способами

4.1 Фильтрация по варианту

Поскольку вариант задания — **Go (Golang) разработчик**, была применена следующая логика фильтрации:

- Поиск подстроки `golang` или `goland` в названии вакансии
- Поиск точного совпадения `go`, `golang` или `goland` в массиве навыков

Такой подход исключает ложные срабатывания на слова вроде "going "good "negotiation".

4.2 Собственная реализация MapReduce

Архитектура решения:

1. **Чтение чанками:** Файл читается из S3 частями по 2 MB с использованием HTTP Range Requests

2. **Обработка границ:** Каждый чанк обрезается по последнему символу переноса строки (`\n`), чтобы не разрывать JSON-объекты

3. **Map-фаза:** Для каждой вакансии извлекаются навыки, приводятся к нижнему регистру, сортируются по алфавиту и генерируются синглтоны, пары и тройки

4. **Reduce-фаза:** Агрегация результатов всех чанков

Ключевые особенности реализации:

- Использование `itertools.combinations` для генерации пар и троек
- Применение `set()` для удаления дубликатов навыков внутри одной вакансии
- Сортировка навыков по алфавиту для обеспечения детерминированности результатов

Топ-5 результатов MapReduce:

Тип	Навык(и)	Частота
S	golang	104
S	go	64
S	postgresql	60
P	golang, postgresql	45
S	docker	39

Таблица 1: Топ-5 навыков по частоте

4.3 SQL-запрос через DuckDB

DuckDB — это встраиваемая аналитическая СУБД, оптимизированная для OLAP-запросов. Она поддерживает прямое чтение данных из S3 через расширение `httpfs`.

Архитектура запроса:

1. **CTE `normalized_skills`:** Нормализация типов данных (преобразование строк в массивы)

2. **CTE `filtered`:** Фильтрация вакансий по варианту (Go)

3. **CTE `unnested`:** Разворачивание массива навыков в отдельные строки

4. **CTE `unique_skills_per_vacancy`:** Удаление дубликатов навыков внутри вакансии (аналог `set()` в Python)

5. **CTE `grouped`:** Группировка и сортировка навыков для генерации пар и троек

6. **Финальный SELECT:** UNION ALL трёх запросов для синглтонов, пар и троек

5 Этап 4. Верификация результатов

5.1 Методология сравнения

Для обеспечения точного сравнения результатов были применены следующие меры:

- **Нормализация ключей:** Все навыки приведены к нижнему регистру и отсортированы по алфавиту
- **Удаление дубликатов:** Использован `SELECT DISTINCT` в DuckDB и `set()` в Python
- **Идентичная логика фильтрации:** Оба подхода используют одинаковые условия отбора вакансий

5.2 Результат верификации

Результаты полностью совпадают!

Все ключи (названия навыков) и их значения (частоты) идентичны в обоих подходах. Это подтверждает корректность реализации MapReduce и правильность SQL-запроса.

6 Анализ роста числа пар и троек навыков

6.1 Комбинаторное объяснение

Рост числа пар и троек навыков объясняется **комбинаторикой**: если в вакансии N навыков, то она генерирует:

- $C(N, 2) = \frac{N!}{2! \times (N-2)!}$ пар
- $C(N, 3) = \frac{N!}{3! \times (N-3)!}$ троек

Например:

- Вакансия с 3 навыками $\rightarrow$ 3 пары и 1 тройка
- Вакансия с 5 навыками $\rightarrow$ 10 пар и 10 троек
- Вакансия с 10 навыками $\rightarrow$ 45 пар и 120 троек

Таким образом, счётчики пар и троек растут **нелинейно** по отношению к числу синглтонов, что и наблюдается в наших результатах.

6.2 Наблюдения из данных

Из топ-5 результатов видно:

- Синглтон `golang` встречается **104 раза**
- Пара `golang + postgresql` встречается **45 раз**

Это означает, что примерно **43%** вакансий с навыком `golang` также содержат `postgresql`. Это логично, так как Go часто используется для backend-разработки, где PostgreSQL является стандартной СУБД.

7 Этап 5. Анализ влияния навыков на заработную плату

7.1 Методология

Для анализа влияния навыков на зарплату был применён следующий подход:

1. Выбран **базовый навык** — самый частый навык в выборке (`golang`, 104 вакансии)
2. Рассчитана **медианная зарплата** для вакансий с базовым навыком
3. Для каждой пары навыков, содержащей базовый, рассчитана медианная зарплата
4. Вычислена **разница** между медианой пары и медианой базового навыка

7.2 Результаты

Топ-10 навыков, влияющих на зарплату:

Базовый навык	Медиана базы	Доп. навык	Медиана пары	Разница (руб.)
golang	200 000	redis	275 000	+75 000
golang	200 000	postgresql	232 375	+32 375
golang	200 000	oon	225 000	+25 000
golang	200 000	sql	225 000	+25 000
golang	200 000	git	225 000	+25 000
golang	200 000	docker	225 000	+25 000
golang	200 000	rabbitmq	219 875	+19 875
golang	200 000	kubernetes	200 000	0
golang	200 000	go	130 000	-70 000
golang	200 000	mysql	102 625	-97 375

Таблица 2: Влияние навыков на зарплату

7.3 Интерпретация результатов

Навыки, повышающие зарплату:

1. **Redis (+75 000 руб.)** — Наибольшую прибавку даёт знание Redis. Это объясняется тем, что Redis используется в высоконагруженных системах, где требуются опытные разработчики. Вакансии с Redis обычно предполагают работу с микросервисной архитектурой и кэшированием, что характерно для senior-позиций.
2. **PostgreSQL (+32 375 руб.)** — Второй по влиянию навык. PostgreSQL является стандартом для enterprise-разработки, и его знание свидетельствует о работе с серьёзными проектами.

3. **Docker, Git, SQL (+25 000 руб.)** — Эти навыки являются базовыми для современного backend-разработчика. Их наличие в паре с Go указывает на полноценный стек разработчика, что повышает ценность кандидата.

Навыки, понижающие зарплату:

1. **MySQL (-97 375 руб.)** — Наибольшее понижение. Это может объясняться тем, что MySQL чаще используется в небольших проектах и стартапах, где зарплаты ниже. Также возможно, что вакансии с MySQL ориентированы на junior-разработчиков.

2. **Go (-70 000 руб.)** — Парадоксальный результат: наличие навыка "go"в паре с "golang"понижает зарплату. Это объясняется тем, что некоторые вакансии указывают оба варианта написания, но при этом предлагают более низкие зарплаты (возможно, это региональные компании или позиции с неполной занятостью).

8 Этап 6. Региональный анализ

8.1 Методология

Для регионального анализа были выбраны три региона:

- **Москва** — крупнейший IT-рынок России
- **Санкт-Петербург** — второй по величине IT-хаб
- **Краснодар** — развивающийся региональный рынок

Для каждого региона рассчитаны:

- Количество вакансий
- Медианная зарплата
- Средняя зарплата

8.2 Результаты

Сравнение регионов:

Регион	Вакансий	Медианная ЗП (руб.)	Средняя ЗП (руб.)
Москва	8	300 000	268 818.75
Санкт-Петербург	5	225 000	191 850.00
Краснодар	2	117 500	117 500.00

Таблица 3: Региональный анализ

8.3 Инсайты и наблюдения

1. **Московский рынок доминирует:** Медианная зарплата в Москве (300 000 руб.) на **33% выше**, чем в Санкт-Петербурге (225 000 руб.), и на **155% выше**, чем в Краснодаре (117 500 руб.). Это соответствует общей тенденции концентрации high-paying IT-вакансий в столице.

2. **Разрыв между медианой и средним:** В Москве средняя зарплата (268 818 руб.) ниже медианы (300 000 руб.), что указывает на наличие вакансий с относительно низкими зарплатами, которые "тянут"среднее вниз. Это может быть связано с наличием junior-позиций или частичной занятостью.

3. **Краснодар как развивающийся рынок:** Несмотря на небольшое количество вакансий (2), Краснодар предлагает конкурентоспособные зарплаты для регионального рынка. Медиана 117 500 руб. может быть привлекательной с учётом более низкой стоимости жизни по сравнению с Москвой и Санкт-Петербургом.

4. **Дефицит Go-разработчиков в регионах:** Общее количество вакансий в регионах значительно меньше, чем в Москве. Это создаёт возможности для удалённой работы или релокации разработчиков.

9 Этап 7. Профилирование и сравнение подходов

9.1 Методология профилирования

Для объективного сравнения производительности каждый скрипт был запущен **3 раза**, после чего рассчитаны:

- Среднее время выполнения
- Пиковое потребление памяти

9.2 Результаты профилирования

Подход	**Среднее время (сек)**	**Пик памяти (MB)**
MapReduce (Python)	2.5654	0.04
DuckDB	2.1809	0.04

Таблица 4: Сравнение производительности

9.3 Анализ результатов

1. **Производительность:** DuckDB работает на **15% быстрее** (2.18 сек vs 2.57 сек). Это объясняется тем, что DuckDB использует векторизованное выполнение запросов и оптимизированный C++ движок, тогда как Python-код интерпретируется и создаёт больше объектов в памяти.

2. **Потребление памяти:** Оба подхода показывают одинаковое потребление памяти (0.04 MB). Это связано с тем, что профилирование через `tracemalloc` измеряет только память, выделенную в основном процессе Python, а DuckDB работает в отдельном процессе. Для более точного измерения памяти DuckDB потребовалось бы использовать системные мониторинговые инструменты (например, `psutil`).

3. **Масштабируемость:** При увеличении объёма данных (например, до 100 000 вакансий) преимущество DuckDB стало бы более заметным, так как его движок оптимизирован для работы с большими объёмами данных и использует параллельное выполнение.

10 Бонусное задание: Оптимизация расчётов

10.1 Применённые оптимизации

В ходе работы были применены следующие оптимизации:

1. **Чтение чанками с обрезкой по границам строк:** Вместо чтения фиксированных блоков байтов, каждый чанк обрезается по последнему символу `\n`, что исключает потерю данных из-за разрыва JSON-объектов.

2. **Удаление дубликатов на этапе Map:** Использование `set()` в Python и `SELECT DISTINCT` в DuckDB предотвращает двойной подсчёт навыков внутри одной вакансии.

3. **Оптимизация SQL-запроса:** Использование CTE (Common Table Expressions) позволяет DuckDB оптимизировать план выполнения запроса и избежать повторных вычислений.

10.2 Сравнение “до” и “после”

К сожалению, в рамках данной работы не проводилось измерение производительности “до” оптимизации, так как оптимизации были применены на этапе разработки. Однако можно отметить, что финальная версия кода работает стабильно и выдаёт корректные результаты.

11 Выводы

11.1 Технические выводы

1. **Оба подхода (MapReduce и DuckDB) дают идентичные результаты**, что подтверждает корректность реализации.

2. **DuckDB показывает лучшую производительность** (на 15% быстрее) благодаря оптимизированному движку.

3. **Формат JSONL** является оптимальным для аналитических задач, так как позволяет читать данные частями.

4. **Нормализация данных** (приведение к нижнему регистру, удаление дубликатов, сортировка) критически важна для получения корректных результатов.

11.2 Бизнес-выводы

1. **Наиболее востребованные навыки для Go-разработчиков:** golang (104 вакансии), go (64), postgresql (60), docker (39).
2. **Наиболее прибыльные комбинации навыков:** Go + Redis (+75 000 руб.), Go + PostgreSQL (+32 375 руб.).
3. **Региональные различия:** Москва предлагает зарплаты на 33-155% выше, чем другие регионы.
4. **Рекомендации для разработчиков:** Для максимизации зарплаты следует изучать Redis, PostgreSQL, Docker и Kubernetes.

11.3 Рекомендации для дальнейших исследований

1. **Расширить выборку:** Проанализировать данные за более длительный период (например, 6 месяцев) для получения более репрезентативных результатов.
2. **Добавить временной анализ:** Изучить, как меняется спрос на навыки и уровень зарплат со временем.
3. **Кластеризация вакансий:** Применить методы машинного обучения для группировки вакансий по схожим требованиям.
4. **Анализ текста описаний:** Использовать NLP для извлечения дополнительной информации из описаний вакансий.

12 Список использованных технологий

- **Python 3.12** — основной язык программирования
- **boto3** — библиотека для работы с S3
- **DuckDB** — встраиваемая аналитическая СУБД
- **Moto Server** — S3-совместимое хранилище для тестирования
- **pandas** — библиотека для работы с табличными данными
- **itertools** — стандартная библиотека для генерации комбинаций

13 Заключение

В ходе выполнения данной работы был успешно реализован полноценный аналитический пайплайн для обработки данных о вакансиях. Проведено сравнение двух подходов к обработке данных — собственной реализации MapReduce и использования DuckDB. Оба подхода показали идентичные результаты, что подтверждает корректность реализации. DuckDB продемонстрировал преимущество в производительности на 15%.

Выполнен анализ влияния навыков на заработную плату Go-разработчиков и проведён региональный анализ. Полученные результаты могут быть полезны как для разработчиков при планировании карьеры, так и для HR-специалистов при формировании предложений о работе.

Все результаты получены на собственных запусках скриптов.